



EENG 350: EEID Lab

Mini Project: Computer Vision and System Integration

Prachi Sharma



This slide deck has been redesigned for clarity using a genAI tool

Mini Project Assignment set-up

Use the camera as a user interface: the marker's position sets each wheel's goal.

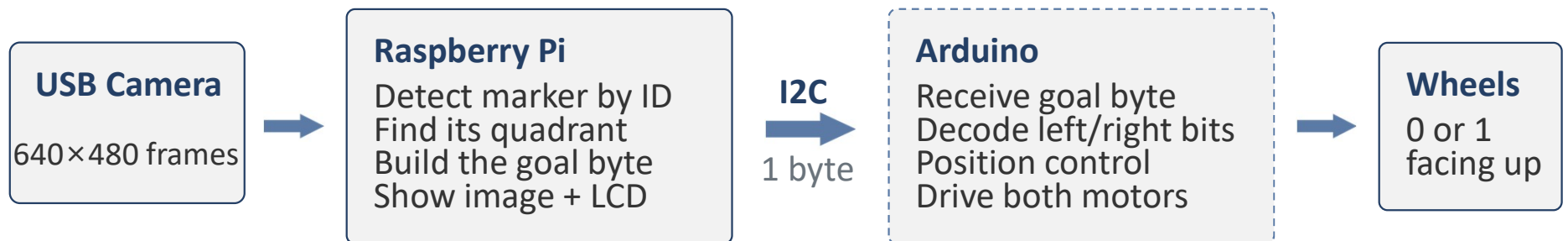
- Two tape marks per wheel, 180° apart, labeled 0 and 1
- At power-up, both wheels start with 0 facing up
- The marker's image quadrant sets each wheel's goal
- The controller holds position if you twist a wheel by hand (integral control)
- The Pi shows the live image and where the marker is.
- The LCD shows Goal Position: L R (left wheel 0/1, right wheel 0/1)

Quadrant	Left Wheel	Right Wheel
NE	0	0
NW	0	1
SW	1	1
SE	1	0

Quadrant → wheel goal, from the handout

System Architecture

- Vision and decision-making is done on the Pi
- Real-time control is implemented on the Arduino
- I2C communication between Pi(leader) and Arduino(follower)



Step 1: Set the Target and Camera

```
import cv2
import numpy as np

TARGET_ID = 0          # marker to track
ARUCO_DICT = cv2.aruco.DICT_6X6_50
CAMERA_INDEX = 0
FRAME_WIDTH, FRAME_HEIGHT = 640, 480
DEADBAND_PX = 15      # jitter near center lines

QUADRANT_BITS = {
    "NE": (0, 0),
    "NW": (0, 1),
    "SW": (1, 1),
    "SE": (1, 0),
}
```

- The dictionary must match the printed marker, or nothing is detected.
- 640×480 keeps detection fast as discussed in Assignment 2 but check the camera you are using. This might not work with the newer Logitech cameras.
- Markers with ID different than the TARGET_ID are ignored

Step 2: Build the ArUco Detector

```
def make_detector():  
    d = cv2.aruco.getPredefinedDictionary(ARUCO_DICT)  
    p = cv2.aruco.DetectorParameters()  
    det = cv2.aruco.ArucoDetector(d, p)  
    def detect(gray):  
        corners, ids, _ = det.detectMarkers(gray)  
        return corners, ids  
    return detect
```

Color does not add any information to the marker so the run detection in grayscale.

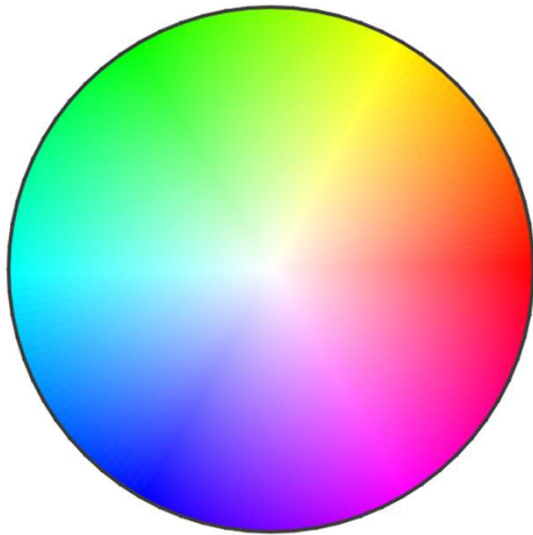
Look at Assignment 2 slides for “Conversion to Grayscale”

Step 3: Find One Specific Marker

- Define a function that will scan every detected ID and keep only the one you are tracking.
- In this function you should also do the following:
 - If no marker is seen then id is None
 - Other marker IDs in the frame are simply ignored
 - Find the center of the marker: Center = average of the four corners
 - Corners come back as $(x, y) = (\text{column}, \text{row})$. Note that this is the reverse of `image[row, col]`

From Assignment 2: Detecting Colors in OpenCV

OpenCV stores color images as Blue-Green-Red(BGR), not RGB.



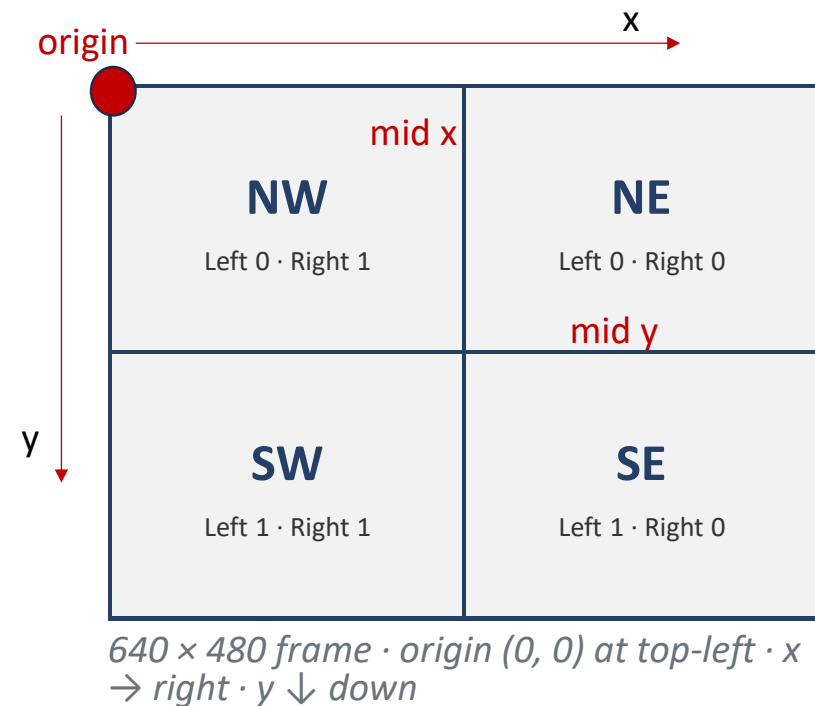
- An image is a 3-D NumPy array: `img[row, col]` → that pixel's [B, G, R]
- Indexing is (y, x) — row before column — the opposite of the usual math (x, y)
- The origin (0, 0) is the top-left corner; down and right are positive

```
import cv2
import numpy as np

imagebgr = cv2.imread('color-wheel.png')
y, x, c = imagebgr.shape
myBGR = imagebgr[int(y/4), int(x/4)] # pixel
from center of upper-left quadrant
print("BGR:", myBGR)

imagehsv = cv2.cvtColor(imagebgr,
cv2.COLOR_BGR2HSV)
myHSV = imagehsv[int(y/4), int(x/4)]
print("HSV:", myHSV)
```

Step 4: Map the Marker Position to a Quadrant



- y grows downward, so north means small y
- East is the right side of the displayed image. Your camera view could be mirrored
- Use the frame's actual size, not a hard-coded 640×480

Step 5: Pack quadrant information into one byte

Quadrant	Left	Right	Binary
NE	0	0	00
NW	0	1	01
SE	1	0	11
SW	1	1	10

- We have defined QUADRANT_BITS at the start of the code as a look up table from the handout.
- From Step 4 we will get the quadrant as shown in column 1 of the table.
- You can use left shift to combine the left and right wheel positions into a two-bit binary value.
- Make sure to add code (in main loop) so that the byte is transferred only when there is a change in the marker's quadrant and not every frame.

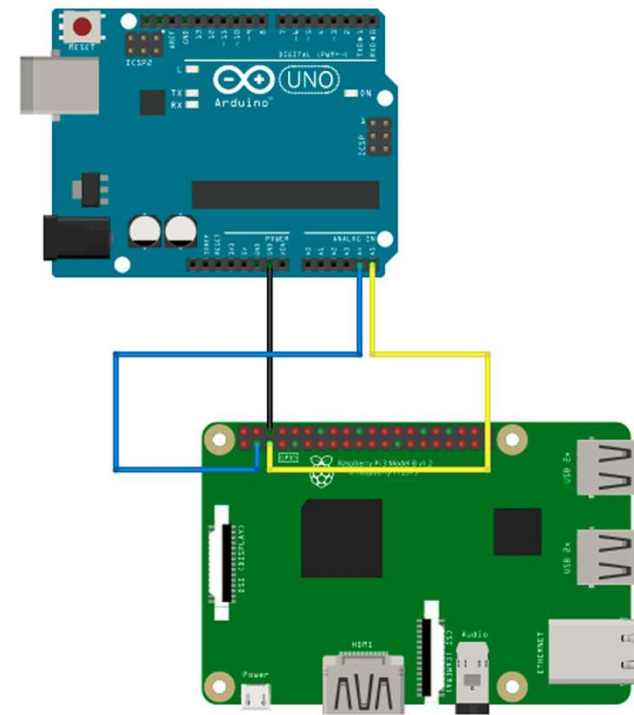
I2C Wiring: Pi as Master, Arduino at Address 8

You will use three wires to connect the pi to the arduino: two signals and a shared ground.

Signal	Raspberry Pi	Arduino Uno
SDA	GPIO2 (pin 3)	A4
SCL	GPIO3 (pin 5)	A5
Ground	GND (pin 6)	GND

Arduino will join the bus with `Wire.begin(8)`

- For I2C to work reliably, common ground is required

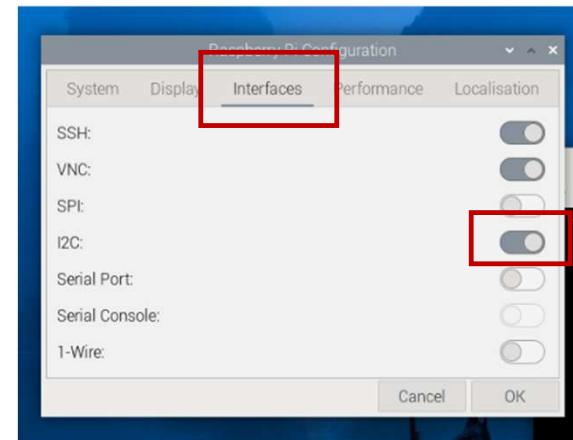
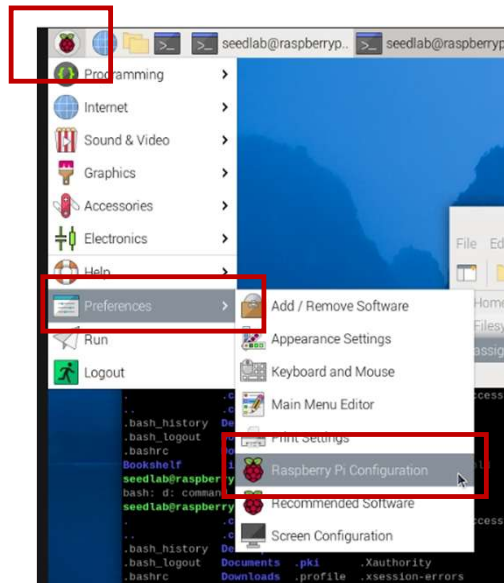


Black is GND, yellow is SCL, and blue is SDA.

Enable I2C on the Pi

On the Pi:

- Open the Menu → Preferences → Raspberry Pi Configuration
- Go to the Interfaces tab
- Turn I2C on.



Check I2C on the Pi

Connect the Arduino and run a simple sketch for `wire.begin(8)`. Do this once per Pi, before writing any communication code.

```
# Scan bus 1 with the Arduino sketch running
i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:      -- -- -- -- -- 08 -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- --
```

- 08 in the grid means the Arduino answered
- If you do not see 08: check wiring, ground, and that the sketch running on the Arduino called `Wire.begin(8)`
- Now you can write a byte of data using `smbus2`. You can explore `write_byte()` instead of `write_byte_data()`
- Arduino will receive the data and we can print it on the serial monitor to check if data was transmitted correctly.
- The Arduino should use this data for motor control

What you see	What it means
Nothing prints	Writes aren't reaching the Arduino: run <code>i2cdetect</code> , check the ground
Two bytes, the first one 0	The Pi is sending a register offset (<code>write_byte_data</code>)
A value above 3	Goal packing doesn't match between Pi and Arduino

Integration Debugging Checklist

What you see	Check first
No marker detected	Dictionary matches the printout; exposure (v4l2-ctl --all); whole marker in frame
Wrong quadrant	x and y swapped? y grows downward; camera view mirrored?
Pi prints "I2C write failed"	i2cdetect -y 1 shows 08? Common ground? Sketch running?
Arduino receives two bytes	write_byte_data used instead of write_byte
Camera window stutters	Slow LCD writes on the main thread — use threading

Make sure you test one link at a time